Travaux dirigés : créer la table et le modèle des documents

Objectif

Dans cet exercice, vous allez préparer la structure qui permettra à Laravel d'enregistrer des informations dans PostgreSQL. À la fin du TD, vous aurez créé la table `documents`, ajouté un document et relu son contenu avec Eloquent.

Une **migration** est un fichier PHP qui décrit une modification de la structure d'une base de données, par exemple la création d'une table. Un **modèle Eloquent** est une classe PHP qui permet à Laravel de travailler avec les lignes de cette table. Le formateur présente ces notions avant le TD.

Suivez les étapes dans l'ordre. Les commandes s'exécutent dans le terminal intégré de VS Code, dans PowerShell ou dans un terminal WSL. PHP, Artisan et PostgreSQL s'exécutent dans Docker : il n'est pas nécessaire de les installer sur Windows.

Avant de commencer

- Le projet Laravel a déjà été installé en suivant le guide de démarrage Docker. Docker Desktop est lancé et opérationnel.
- Ouvrez dans VS Code le dossier du projet Laravel, celui qui contient `artisan` et `compose.yaml`.
- Dans VS Code, ouvrez **Terminal > Nouveau terminal**. Toutes les commandes `docker compose` de ce guide se lancent depuis ce dossier.
- Partez de la branche `jour2-decouverte-laravel`. La branche `jour2-postgresql-migrations-modeles` contient la correction : ne la prenez pas comme point de départ pour créer les fichiers vous-même.

Vérifiez votre branche :

```
git branch --show-current
```

Si nécessaire, changez de branche :

```
git switch jour2-decouverte-laravel
```

À vérifier : le nom affiché est `jour2-decouverte-laravel`. Si Git refuse de changer de branche parce que vous avez des modifications locales, ne les supprimez pas : demandez l'aide du formateur. Si la branche est introuvable dans votre copie du dépôt GitHub, récupérez la liste des branches avec `git fetch origin`, puis réessayez `git switch jour2-decouverte-laravel`.

> Le nom exact du fichier de migration sera différent sur chaque poste : Laravel ajoute la date et l'heure de sa création. Repérez sa fin, `create_documents_table.php`.

1. Vérifier Docker et la connexion à PostgreSQL

Dans le terminal, démarrez les services s'ils ne fonctionnent pas déjà :

```
docker compose up -d
```

Vérifiez leur état :

```
docker compose ps
```

À vérifier : les services `app` et `db` sont démarrés et `db` indique `healthy`. Si le premier démarrage n'est pas terminé, reprenez le guide Docker avant de continuer.

Vérifiez la connexion utilisée par Laravel :

```
docker compose exec app php artisan db:show
```

À vérifier : la connexion indique PostgreSQL, l'hôte `db` et la base `formation_documents`. Le nom `db` est celui du service PostgreSQL dans le réseau Docker. Le volume Docker conserve les données de la base lors d'un arrêt ordinaire des conteneurs.

Le fichier `.env`, à la racine du projet, contient la configuration locale de Laravel. Par exemple, `DB_CONNECTION=pgsql`, `DB_HOST=db` et `DB_PORT=5432` indiquent le type de base et son adresse dans Docker. Le fichier `.env.example` sert de modèle pour une nouvelle installation. **Ne remplacez pas votre `.env` existant et ne partagez pas ses identifiants.**

2. Générer le modèle et sa migration

Lancez cette commande depuis le dossier du projet :

```
docker compose exec app php artisan make:model Document --migration
```

`make:model Document` crée le modèle ; `--migration` demande également la création d'une migration associée.

À vérifier dans l'explorateur de fichiers de VS Code :

- Le fichier `app/Models/Document.php` est apparu.
- Un nouveau fichier dont le nom se termine par `create_documents_table.php` est apparu dans `database/migrations/`.

Ne relancez pas la commande si ces fichiers existent déjà. Si Laravel indique que le modèle existe, vérifiez la branche et demandez l'aide du formateur avant de modifier ou supprimer des fichiers.

3. Décrire la table dans la migration

Dans VS Code, ouvrez `database/migrations/`, puis ouvrez **le fichier que vous venez de créer** : son nom se termine par `create_documents_table.php`.

Remplacez tout le contenu de ce fichier par le code suivant, puis enregistrez-le avec **Ctrl+S** :

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('documents', function (Blueprint $table) {
            $table->id();
            $table->string('title', 100);
            $table->string('category', 50);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('documents');
    }
};
```

La méthode `up()` décrit la création de la table. `id()` ajoute son identifiant ; `title` contient le titre, limité à 100 caractères ; `category` contient la catégorie, limitée à 50 caractères. `timestamps()` ajoute les colonnes `created_at` et `updated_at`.

La méthode `down()` décrit l'opération inverse : ici, supprimer la table. Elle est utilisée lors d'un retour arrière et peut donc supprimer les données qu'elle contient. Nous ne lancerons pas ce retour arrière dans cet exercice.

À vérifier : le fichier est enregistré et contient bien `Schema::create('documents', ...)`. À ce stade, écrire le fichier ne crée pas encore la table : il reste à exécuter la migration.

4. Préparer le modèle Document

Dans VS Code, ouvrez `app/Models/Document.php`. Remplacez tout son contenu par le code suivant, puis enregistrez le fichier :

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Attributes\Fillable;
use Illuminate\Database\Eloquent\Model;

#[Fillable(['title', 'category'])]
class Document extends Model {}
```

Par convention, le modèle `Document` représente les lignes de la table `documents`. L'attribut `Fillable` autorise l'affectation groupée des champs `title` et `category`, par exemple avec `Document::create(...)`. Il ne remplace pas la validation des données, qui sera ajoutée lors de la construction de l'API.

À vérifier : le nom de la classe est `Document` et les deux noms de champs correspondent exactement aux colonnes de la migration.

5. Exécuter les migrations

Vérifiez une dernière fois que vous travaillez bien sur la base locale de formation, puis lancez :

```
docker compose exec app php artisan migrate
```

Cette commande exécute **toutes les migrations en attente**, y compris les trois fichiers fournis par Laravel pour les utilisateurs, le cache et les tâches, ainsi que votre nouvelle migration des documents. Une migration déjà exécutée ne sera normalement pas rejouée.

À vérifier : la sortie affiche `create_documents_table` avec le statut `DONE`. S'il n'apparaît pas, consultez l'état à l'étape suivante ou demandez au formateur. Ne lancez pas `migrate:fresh`, `migrate:reset` ou `migrate:rollback` pour résoudre une erreur : ces commandes peuvent supprimer des tables ou des données.

6. Vérifier la migration et la table

Consultez l'état des migrations :

```
docker compose exec app php artisan migrate:status
```

À vérifier : `create_documents_table` apparaît avec le statut `Ran`. Avant le tout premier `migrate`, Laravel peut indiquer que sa table de suivi des migrations n'existe pas encore ; après une exécution réussie, elle est créée.

Inspectez ensuite la structure de la table :

```
docker compose exec app php artisan db:table documents
```

À vérifier : retrouvez les colonnes `id`, `title`, `category`, `created_at` et `updated_at`. Cette commande décrit la structure ; elle ne liste pas les documents enregistrés.

Dans pgAdmin, ouvrez **Servers > PostgreSQL formation > Databases > formation_documents > Schemas > public > Tables**. Faites un clic droit sur **Tables > Refresh** : la table `documents` doit apparaître. Pour la connexion à pgAdmin, consultez l'étape 8 du guide de démarrage.

7. Créer et relire un document avec Tinker

Tinker est une console interactive qui permet d'essayer du code PHP dans le contexte de l'application Laravel. Lancez-la dans le conteneur :

```
docker compose exec app php artisan tinker
```

Une fois l'invite de Tinker affichée, copiez-collez ce code **dans Tinker**, puis appuyez sur Entrée :

```
\App\Models\Document::create([
    'title' => 'Contrat de formation',
    'category' => 'Administratif',
]);
```

À vérifier : Tinker affiche un objet `Document` contenant notamment un `id`, le titre et la catégorie. La ligne a été enregistrée dans PostgreSQL. Relancer `create` ajoute un nouveau document.

Toujours dans Tinker, lisez les documents :

```
\App\Models\Document::all();
```

À vérifier : la collection affichée contient le document ajouté. Les dates `created_at` et `updated_at` ont été renseignées par Eloquent.

Pour voir la même ligne dans pgAdmin, faites un clic droit sur la table **documents**, puis choisissez **View/Edit Data > All Rows**. Le titre et la catégorie du document apparaissent. Si la grille était déjà ouverte, relancez l'affichage pour l'actualiser.

Quittez Tinker pour revenir au terminal :

```
exit
```

La route d'exemple `/api/exemples/document` renvoie toujours des données fictives : elle n'est pas encore reliée à cette table. L'API CRUD sera construite à l'étape suivante.

8. Questions de compréhension

1. Quel fichier décrit les colonnes de la table `documents` ?
2. Quelle est la différence entre écrire une migration et exécuter `migrate` ?
3. Quel rôle joue le modèle `Document` ?
4. Que permettent les champs déclarés dans `Fillable` ?
5. Que se passe-t-il si vous exécutez à nouveau `\App\Models\Document::create(...)` ?

Discutez des réponses avec le formateur, puis comparez votre travail avec la branche de correction `jour2-postgresql-migrations-modeles` selon ses indications. Ne changez pas de branche sans avoir d'abord sauvegardé votre travail : Git peut refuser le changement si les fichiers entrent en conflit.

En cas de difficulté

Situation	Vérification
`no configuration file provided`	Le terminal est-il dans le dossier contenant `compose.yaml` ?
Docker ne répond pas	Docker Desktop est-il démarré ? Vérifiez avec `docker compose ps`.

Situation	Vérification
Le service `app` n'est pas actif	Consultez `docker compose logs app` et reprenez le guide de démarrage si les dépendances ne sont pas installées.
Connexion à PostgreSQL impossible	Vérifiez `docker compose ps`, l'état `healthy` de `db` et la configuration `DB_*` locale, sans partager les mots de passe.
Modèle ou migration déjà présent	Vérifiez la branche et demandez au formateur avant de supprimer quoi que ce soit. Ne générez pas de deuxième migration `create_documents_table`.
Table `documents` absente	Vérifiez que la migration a été enregistrée, puis consultez `migrate:status` et le message de `migrate`.
Erreur PHP ou `Class not found`	Vérifiez le chemin des fichiers, le nom `Document`, les imports et les points-virgules ; enregistrez les fichiers puis réessayez.
`MassAssignmentException`	Vérifiez l'attribut `#[Fillable(['title', 'category'])]` et l'import `Fillable` dans le modèle.

Le document de cet exercice est une ligne d'informations : aucun fichier n'est téléversé ou stocké à cette étape.